

Speech notes — 15 minutes

Towards Robust Uzbek Neural
Dependency Parsing

Cross-Treebank Training & Linguistically-Motivated Subword Fusion

International Conference on Applications of Natural Language
to Information Systems

NLDB 2026 — 18 June 2026

Sanatbek Matlatipov

National University of Uzbekistan, Tashkent

`s.matlatipov@nuu.uz`

How to use these notes. Each section maps to one slide. The bracketed times [start → end] are cumulative running time. Phrases in **bold** are the ones to emphasise; [pause] marks a short breath. Plain text is the spoken script — read naturally, not word-for-word.

Total budget: ≈15 minutes of speech inside a 15-minute slot. Target around 0:30–0:45 per slide; the motivation now spans two slides — **Slide 4 (challenges)** and **Slide 5 (suffixes carry the syntax)** — so keep each tight. Slow down on **Slide 14 (fusion)** and **Slides 18–19 (results)**.

Two ideas the audience must remember: last-subword fusion and cross-treebank training. **Numbers to land:** 85.08 / 72.39 / 63.81 (UPOS / UAS / LAS) and cross-treebank **+9.62 to +11.16 LAS**. Also mention: **Uzbek is spoken by 40 million, Turkic Family, SOV word order, null subject, no articles.**

Slide 1 — Title

[0:00 → 0:20 min]

Good morning / afternoon, and thank you to the NLDB organisers.

My name is **Sanatbek Matlatipov**, from the National University of Uzbekistan. My talk is on **Towards Robust Uzbek Neural Dependency Parsing** — making Uzbek parsing robust in a genuinely **low-resource** setting, using two lightweight and efficient ideas rather than bigger models.

Slide 2 — Outline

[0:20 → 0:35 min]

The talk has a simple arc:

- Quick **motivation** and related work.
- The **dataset** we built and pair with.
- The **methodology** — pipeline, representation, model.
- And **results**, error analysis, and takeaways.

Slide 3 — Uzbek Language I

[0:35 → 1:05 min]

Before diving into the parsing challenge, let me briefly introduce Uzbek.

Uzbek NLP has seen **growing interest** in recent years — TahrirchiBERT, BERTbek, and several shared tasks demonstrate that. But compared to major world languages, computational resources remain sparse, and we still face genuinely low-resource conditions for structured prediction tasks like parsing.

Uzbek is the **official language** of Uzbekistan. It is native to Central Asia, Afghanistan, parts of Russia, and Xinjiang in China. Roughly **40 million** people speak Uzbek as of 2026 — a significant speech community, yet under-resourced in computational linguistics.

The map on the right shows where Uzbek is spoken. You can see it spans a large geographic region across Central Asia and beyond.

Slide 4 — Why is Uzbek dependency parsing hard?

[1:05 → 1:35 min]

Three things make Uzbek parsing hard. First, Uzbek is **agglutinative** — a single word stacks suffixes for case, tense, person, even evidentiality. Look at the figure: *kelmaganlardanmisizlar?* is **one** Uzbek word, but it unpacks into the eight-word English question “Are you one of those who didn’t come?” — come, not, past, plural, one of, question, are, you. That is the morphological density a parser has to decode. Second, **flexible word order** together with strong genre variation. Third, it is genuinely **low-resource** — fewer than a thousand annotated UD sentences before this work.

On the right are the **Turkic-family** traits the parser must cope with: **null subject** — you can drop the subject pronoun; **no articles** and **no grammatical gender**, unlike English; and **Subject-Object-Verb** order, quite different from English SVO.

[pause]

So the picture is **rich morphology** on one side, **scarce data** on the other. Those are exactly the two problems we attack — better representations for the morphology, more data for the scarcity. The next slide shows **why** that morphology matters so much.

Slide 5 — Suffixes carry the syntax

[1:35 → 2:05 min]

Here is the intuition in one example. Take *bolalarning* — “of the children.” Stem *bola*, plus plural *-lar*, plus genitive **-ning**. That final suffix **is** the syntactic relation: the **-ning** genitive is what creates the possessive arc — **nmod:poss** — onto *kitobi*, and that phrase is the subject of *keldi*, “arrived.”

[pause]

The take-away: **strip the rightmost suffix and the syntax disappears**. A parser that ignores it misses the parse. That is precisely what motivates our **last-subword fusion** later in the talk. The tree is a real example from our treebank.

Slide 6 — The gap and our approach

[2:05 → 2:35 min]

The core problem: existing Uzbek treebanks are small, and neural parsers need **both** more data **and** better representations.

- On the **data** side — a new treebank, **UzUDT**, 684 gold sentences, the largest for Uzbek.
- On the **modelling** side — two lightweight strategies: **last-subword fusion** and **cross-treebank training**.

Crucially: **no heavy architecture changes**. This is the efficient, low-resource-friendly path.

Slide 7 — Contributions

[2:35 → 3:10 min]

Four contributions:

- **UzUDT** — built in INCEpTION, six annotators, full adjudication.
- **Last-subword fusion** — a linguistically-motivated WordPiece-to-UD-token alignment.
- **Cross-treebank training** — merging two genre-complementary Uzbek treebanks.
- A clean **factorial study** — three configs × two data settings = six runs — plus released code and twelve checkpoints.

Headline: best system **85.08 UPOS**, **72.39 UAS**, **63.81 LAS** — and cross-treebank alone adds **nine to eleven LAS points**.

Slide 8 — Where this work fits

[3:10 → 3:45 min]

Briefly — the framework is **Universal Dependencies**; we build on morphology-aware parsing for Turkic languages like Turkish. For Uzbek specifically there was one small treebank, **Uzbek-UT**; we add **UzUDT**.

[pause]

Architecturally we use a standard neural pipeline — Stanza-style, with a **Dozat–Manning biaffine parser** — and Uzbek encoders like **TahrirchiBERT**. Our niche

is **lightweight** robustness: suffix-aware fusion plus cross-treebank data, not a new architecture.

Slide 9 — UzUDT: a new gold Uzbek UD treebank

[3:45 → 4:30 min]

UzUDT: **684 sentences**, about **7,800 tokens**, across fiction and academic text. Annotated in INCEpTION by six people — four linguists and two NLP engineers — with double annotation and **full adjudication**. Full UD v2 layers: UPOS, XPOS, lemma, features, and dependency relations.

[pause]

Inter-annotator agreement is strong — **kappa of 95 on lemmas, 94 on UPOS, 91 on features**. So the labels are reliable. To our knowledge this is the **largest publicly available Uzbek UD treebank**.

Slide 10 — Second treebank & data settings

[4:30 → 5:05 min]

The second treebank is **UD_Uzbek-UT** — 500 sentences, news and fiction, semi-automatic with manual correction.

- Setting **.1** is UzUDT only — 451 training sentences.
- Setting **.2** merges both — 781.

So merging nearly **doubles** the training data. That is a direct test: **is raw data quantity the bottleneck?**

Slide 11 — Why the treebanks are complementary

[5:05 → 5:45 min]

These two treebanks cover **different syntax** — which is exactly why merging helps.

- UT is news-heavy, so many **proper nouns** — 5.2% versus our 0.3%.
- UzUDT has more **pronouns** and **four times** more adverbial clauses.
- And UT has **ten times** more light-verb compounds.

So they fill each other's gaps — merging is not just more data, it is **broader** data.

Slide 12 — Pipeline: from text to a labeled tree

[5:45 → 6:30 min]

Here is the full pipeline, left to right. A UD-tokenised sentence goes into a **frozen encoder** — FastText or TahrirchiBERT. Then our **fusion** step maps subwords to UD tokens. Then a **joint tagger** predicts UPOS, XPOS and features. Then a **biaffine parser** produces the labeled tree.

[pause]

Two design choices matter. The encoder is **frozen** — efficient. And training is **sequential**: tagger first, then the parser runs on **re-tagged** data — the dashed arrow — so there is no gold-tag leakage at inference.

Slide 13 — Token representation: four channels

[6:30 → 7:05 min]

Each token is a concatenation of four vectors: a static **word** embedding, a **character-LSTM** vector, the **predicted UPOS** embedding, and a **contextual** vector from BERT.

[pause]

Two of the six runs differ here. In the **FastText** config, the word channel is static 300-d and there is **no** contextual channel. In the **TahrirchiBERT** config, that contextual channel is a **768-d fused** vector — and that fusion is our key contribution.

Slide 14 — Subword→token fusion: our key heuristic

[7:05 → 8:05 min]

This is the heart of the method. BERT emits **WordPieces**, but UD annotates **tokens** — they do not line up. We must collapse several subword vectors into one per token.

[pause]

Our heuristic — **last-subword**: take the vector of the **final** WordPiece as the token's representation. For *bolalarning* that is *##ning*, the orange piece.

[pause]

Why the last one? Because Uzbek is **suffixing** — the rightmost piece carries the **outermost** morphology. *##ning* literally is the genitive case — the bit the parser most needs. The baseline, **mean pooling**, averages all pieces and **dilutes** exactly that suffix signal. This one-line change is worth almost **four LAS points**.

Slide 15 — Joint morphosyntactic tagger

[8:05 → 8:45 min]

The tagger is a shared **BiLSTM** over the token vectors, feeding **three softmax heads** — UPOS, XPOS and morphological features — jointly. Joint tagging lets the three layers share evidence, which helps when data is scarce.

[pause]

And only the **predicted** UPOS embedding — not the gold tag — is exported downstream to the parser. Again, **no leakage**.

Slide 16 — Deep biaffine dependency parser

[8:45 → 9:25 min]

Standard **Dozat–Manning** biaffine parser. The BiLSTM states — now including the predicted UPOS embedding — are projected by MLPs into **head** and **dependent** subspaces. A biaffine attention then scores **every** head–dependent pair, and we decode a labeled tree over the UD tokens.

[pause]

The example on the right — *asarlal bizga eshigini ochib beradi* — shows subject, oblique, the **xcomp**, the object, and the root.

Slide 17 — Experiment matrix

[9:25 → 10:00 min]

Six runs, fully factorial. Rows vary three things:

- **E1 vs E2** — the embedding: static FastText versus contextual BERT.

- **E2 vs E3** — the fusion: our last-subword versus mean pooling.
- **.1 vs .2** — the data: single versus cross-treebank.

The green row, **E2.2**, is the full system: BERT, last-subword, merged data.

Slide 18 — Main results (test set)

[10:00 → 11:00 min]

Tagging on the left, parsing on the right; the green row is our full system, **E2.2**.

- E2.2 wins **four of five** metrics — best UPOS at **85.08**, best features, best UAS **72.39**, best LAS **63.81**.
- The only exception is **XPOS**, where mean pooling edges ahead — a minor, fine-grained-tag effect.

Notice the LAS column climbs steadily as we add **contextual embeddings**, then **fusion**, then **data**.

Slide 19 — Three findings

[11:00 → 12:00 min]

- **1. Contextual beats static.** BERT over FastText gives nearly **five UPOS points** on merged data.
- **2. Fusion matters for parsing.** Last-subword over mean is **+3.76 LAS**. And the ordering is telling: mean-pooled BERT (60.05) actually **loses** to FastText (62.40); last-subword BERT (63.81) wins. A bad fusion wastes your encoder.
- **3. Cross-treebank is the biggest lever.** **+9.62 LAS** for BERT, **+11.16** for FastText — just from merging.

For low-resource parsing, **broader data beats clever modelling**.

Slide 20 — Error analysis (100 test sentences, E2.2)

[12:00 → 12:45 min]

We hand-checked 100 test sentences. The errors concentrate in **attachment**:

- **Prepositional / oblique attachment** — 28%.
- **Coordination scope** — 22%, then subordinate clauses and compounds.

The dominant confusion is **obl versus nmod**, and coordination scope. The tree on the right shows a missed **light-verb construction** — gold **compound:lvc** in blue, the model's wrong arcs dashed in red. Rare constructions are exactly what the smaller treebank under-covers.

Slide 21 — Conclusion

[12:45 → 13:30 min]

Three takeaways:

- We released **UzUDT**, the largest gold Uzbek UD treebank.
- **Cross-treebank training** is the dominant factor — up to eleven LAS points.
- **Last-subword fusion** is a cheap, linguistically-motivated win — almost four LAS points.

Best system: **85.08 / 72.39 / 63.81**. Limitations: small dev sets and some genre mismatch. Next: **BERTbek** and **BERT-plus-FastText fusion**. Everything — data and twelve checkpoints — is on Hugging Face, code on GitHub.

Slide 22 — Thank you

[13:30 → 14:00 min]

That brings me to the end. I would like to thank the open-source teams behind **Stanza**, **TahrirchiBERT**, and **Universal Dependencies**, and the UzUDT annotation team.

[pause]

Thank you very much — I am happy to take your questions. (Contact: s.matlatipov@nuu.uz.)

Appendix backups — only if asked

- **A1 — Parser training dynamics.** Dev LAS/UAS over training; the merged setting converges higher and more stably than UzUDT-only.
- **A2 — Tagger training dynamics.** UPOS / XPOS / UFeats / AllTags F1 across the six runs; E2.2 leads on most curves.
- **A3 — A correctly parsed sentence.** A clean example the full system gets right end-to-end.

Likely Q&A — quick cues

Q. Why freeze the encoder instead of fine-tuning?

Efficiency and stability. On fewer than a thousand sentences, fine-tuning a Transformer overfits; freezing gives a strong, reproducible baseline at a fraction of the cost.

Q. Why last-subword pooling rather than mean or attention?

It is **motivated by Uzbek morphology** — the rightmost subword carries the outermost, syntactically decisive suffix. Empirically it is **+3.76 LAS** over mean pooling, with no extra parameters.

Q. Is last-subword fusion Uzbek-specific?

It targets **suffixing, agglutinative** morphology, so it should transfer to other Turkic languages — Turkish, Kazakh, Uyghur. Testing that is future work.

Q. Why not mBERT, XLM-R, or UDify?

We deliberately compare **lightweight, Uzbek-specific** encoders for a fair low-resource baseline. Multilingual and cross-lingual transfer is a natural — but separate — next step.

Q. The dev sets are tiny (45 / 78) — isn't that noisy?

Yes, and we state it as a limitation. But the headline data effect — **nine to eleven LAS points** — is far larger than any plausible dev-set noise, so the conclusion is safe.

Q. Why does XPOS regress under last-subword?

Mean pooling slightly **smooths fine-grained tags**. The net effect across the other four metrics still favours last-subword, and it is our best overall system (four of five metrics).

Q. Does merging help only because of size, or genre too?

Both. Merging doubles the data *and* the treebanks are genre-complementary — UT brings proper nouns and light-verb compounds, UzUDT brings pronouns and adverbial clauses.

Q. Will the code and checkpoints be released?

Yes — the UzUDT data and **twelve checkpoints** are on Hugging Face, and the full code is on GitHub.